SMART-X IOT PROJECT COMPLETION REPORT

Prog7312

Matteo Nusca
ST10440432 | Group 1

Contents

1. Summary 2
2. IoT context: why these two features 2
3. Trees: historical telemetry queries 3
   Why AVL, not a plain binary search tree 3
   Keys 3
   The range search 3
   Rebalancing 4
   Measured speed 4
4. Graphs: traversal, shortest paths and the spanning tree 4
   Finding who is connected and who is down 5
   Cheapest route 5
   Minimum spanning tree 5
   The tree is not the same as the cheapest route 6
   Self-healing 7
5. Challenges and how they were overcome 7
   5.1 Time keys that repeat, and ranges that must include both ends 7
   5.2 Keeping the tree valid while old readings are evicted 7
   5.3 A spanning tree is not a shortest-path tree 7
   5.4 Defining "down" 7
   5.5 A mesh that gives failures something to do 8
   5.6 Connecting the C# backend to the Blazor front end 8
   5.7 Shared state across threads 8
6. Key learnings 8
7. Testing and verification 10
8. Technology recommendations and future scaling 12
   Scaling the Part 3 structures 14
9. Feedback incorporated and AI use 14
   Feedback 14
   AI use 14
10. References 16

1. Summary

Part 3 completes Smart-X: sensors now form a mesh that routes readings to the gateway at the lowest power cost and heals around failed nodes, and a balanced tree answers "readings between time A and time B" about 33 to 57 times faster than scanning the whole log. Both are on the new Network Topology page, so all three menu pillars now run in one application.

What was built	Where	Measured result
AVL tree for historical telemetry	`SmartX.Shared/Structures/AvlTree.cs`, `TelemetryHistory`	A 10-minute window touched 74 of 8,690 readings; tree 6 µs against 364 µs for a scan (57×)
Weighted mesh graph with BFS, DFS, Dijkstra, Prim and Kruskal	`SmartX.Shared/Mesh/MeshGraph.cs`	Prim and Kruskal agree on 100 random meshes; Dijkstra matches Floyd-Warshall on 30
Self-healing routes	`MeshTopologyService`	Pump A reroutes from 21 mW to 22 mW when Moisture Sensor 1 fails
Network Topology page	`SmartX.Client/Pages/Topology.razor`	Live map, route comparison, failure simulation, history search
Automated tests and CI	`SmartX.Tests`, `.github/workflows/ci.yml`	132 unit tests pass; 46 of 46 API checks pass

2. IoT context: why these two features

Sensor nodes carry limited, generally irreplaceable power and spend most of their energy on communication, and multi-hop routing that picks the minimum-energy route to the sink saves power, so how data hops through a network decides how long the network lives [1]. Surveys of IoT protocols describe the same pattern: resource-constrained devices, multi-hop delivery through intermediate nodes, and gateways that connect devices to the wider network [2]. Smart-X models this with a graph: each ESP32 is a vertex, each radio link is an edge, and the edge weight is the power a packet costs to cross it.

Two questions follow from that. The first is operational: which route should a reading take to the gateway, and what happens when a node dies? A greenhouse relay that goes silent must not strand the sensors behind it. The second is analytical: an operator investigating an anomaly needs to pull one device's readings for a specific window, for example moisture between 14:00 and 16:00, without paging through the entire history. A time-ordered, self-balancing tree gives that lookup its speed [3].

The mesh is simulated, as the brief allows. Link costs come from where devices are deployed: 3 mW inside one zone, 6 mW between zones of a facility, 12 mW between facilities, plus a fixed 0 to 4 mW offset from a stable hash of the two device ids. The same fleet always produces the same mesh, so results are repeatable.

3. Trees: historical telemetry queries

Every device's readings are stored in an AVL tree keyed by time, and a range query only walks the branches that can contain matches, so it costs O(log n + k) for k results instead of O(n) for the whole log. The tree lives in `SmartX.Shared/Structures/AvlTree.cs` and is used by `TelemetryHistory` as a per-device archive of up to 20,000 readings, beside the Part 2 timeline that feeds the live charts.

Why AVL, not a plain binary search tree

Sensor timestamps arrive in ascending order, which is the worst case for an unbalanced tree: it degrades into a linked list and every search becomes a scan. An AVL tree restores balance after each insert or delete with rotations, so its height stays within about 1.44 log2 n [3]. For roughly 8,700 readings that is at most 18 levels, which the smoke test asserts.

Keys

Two readings can carry the same timestamp, and a tree rejects duplicate keys, so the key is a struct of timestamp plus an ingestion sequence number:

```
public readonly record struct HistoryKey(DateTime Timestamp, long Sequence) :
IComparable<HistoryKey>
{
    public int CompareTo(HistoryKey other)
    {
        int byTime = Timestamp.CompareTo(other.Timestamp);
        return byTime != 0 ? byTime : Sequence.CompareTo(other.Sequence);
    }
}
```

The range search

The recursive search skips a whole subtree whenever the node's key shows nothing in range can be there. It also counts the nodes it touches, which is the number the benchmark reports:

```
int cmpLow  = low.CompareTo(node.Key);   // < 0: node is above the lower bound
int cmpHigh = high.CompareTo(node.Key);  // > 0: node is below the upper bound

if (cmpLow < 0)  CollectRange(node.Left,  low, high, results, ref visited);
if (cmpLow <= 0 && cmpHigh >= 0) results.Add(new(node.Key, node.Value));
if (cmpHigh > 0) CollectRange(node.Right, low, high, results, ref visited);
```

`TelemetryHistory.QueryArchive` turns a time window into a key range by padding the ends with the smallest and largest possible sequence numbers, so every reading at the boundary timestamps is included:

```
var hits = tree.Range(new HistoryKey(low,  long.MinValue),
                      new HistoryKey(high, long.MaxValue), out int visited);
```

Rebalancing

`Rebalance` recomputes the node's height, and if the two sides differ by more than one it applies a single or double rotation. Delete works the same way, replacing a node that has two children with its in-order successor. This is what keeps the archive healthy while retention removes the oldest reading once the cap is reached.

Measured speed

The benchmark endpoint runs the same window against the tree and against a scan of every archived reading, 200 times, and checks both return the same count. A 10-minute window held about 60 readings in every run:

Run	Archive size	Tree (µs)	Full scan (µs)	Speed-up
1	8,646	8.27	333.52	40×
2	8,646	9.41	416.98	44×
3	8,661	5.92	284.94	48×
4	8,690	6.37	363.77	57×

The tree touched 74 nodes of 8,690 in the range query. The speed-up grows with the archive because the scan cost grows linearly while the tree's cost grows with its height.

4. Graphs: traversal, shortest paths and the spanning tree

The mesh is a weighted, undirected graph held as an adjacency list in `MeshGraph`: a dictionary from each node to its neighbours and the `LinkCost` of each link. Neighbour lookup is O(1) and the whole graph costs O(V + E) memory, which suits a sparse radio mesh where each node links to only a few others. A node that is down is treated as removed, so every algorithm below automatically routes around failures.

Algorithm	Method	Question it answers on the mesh	Cost
Breadth-first search	`Bfs`	How many hops is each node from the gateway, and which route has the fewest hops?	O(V + E)
Depth-first search (recursive)	`Dfs`, `AnalyseConnectivity`	Which live nodes can still reach the gateway, and which are cut off in islands?	O(V + E)
Dijkstra with a min-heap	`ShortestPath`	What is the cheapest route in total power from one sensor to the gateway?	O((V + E) log V)
Prim with a min-heap	`MinimumSpanningTree`	Which links keep every node joined for the least total power?	O(E log V)

Algorithm	Method	Question it answers on the mesh	Cost
Kruskal with union-find	`MinimumSpanningTree Kruskal`	The same tree, built a different way, as a cross-check	O(E log E)

The heap under Dijkstra and Prim is a hand-written binary min-heap (`MinHeap.cs`), and link costs are a `LinkCost` struct with overloaded `+` and comparison operators, so the algorithms read like arithmetic (`cost + linkCost`, `candidate < known`) while the type refuses negative or invalid costs.

Finding who is connected and who is down

A recursive depth-first search from the gateway collects every live node it can reach. Any live node it misses is cut off, and a second pass groups those into islands:

```
private void DfsVisit(string node, HashSet<string> visited, List<string> order)
{
    visited.Add(node);
    order.Add(node);
    foreach (string neighbour in _adjacency[node].Keys)
        if (IsAlive(neighbour) && !visited.Contains(neighbour))
            DfsVisit(neighbour, visited, order);
}
```

Cheapest route

Dijkstra's method settles the node with the smallest known total cost and then checks whether going through it gives a neighbour a cheaper route, and the min-heap is how this implementation finds that node quickly [4]. A stale heap entry is skipped if a cheaper route has already replaced it:

```
while (heap.TryDequeue(out string node, out LinkCost cost))
{
    if (cost > distance[node]) continue;          // outdated entry
    if (Comparer.Equals(node, destination)) break;
    foreach (var (neighbour, linkCost) in _adjacency[node])
    {
        if (!IsAlive(neighbour)) continue;        // down nodes are skipped
        LinkCost candidate = cost + linkCost;
        if (!distance.TryGetValue(neighbour, out var known) || candidate < known)
        {
            distance[neighbour] = candidate;
            previous[neighbour] = node;
            heap.Enqueue(neighbour, candidate);
        }
    }
}
```

Minimum spanning tree

Prim grows the tree from the gateway, always adding the cheapest link that reaches a node not yet in the tree [5]. Kruskal sorts all links and keeps each one that does not close a loop, using a union-find to track which nodes are already joined [5]:

```
while (heap.TryDequeue(out var entry, out LinkCost cost))
{
    if (inTree.Contains(entry.Node)) continue;
    inTree.Add(entry.Node);
    parent[entry.Node] = entry.Parent;            // next hop towards the gateway
    total += cost;
    PushLinks(entry.Node, inTree, heap);
}
```

The `parent` map is the routing table: a node's route to the server is found by following parents up to the gateway, which `MstResult.PathToRoot` does recursively.

The tree is not the same as the cheapest route

The spanning tree minimises the power of the whole network, not any one node's path. On the textbook graph used in the tests, the tree costs 39 mW and the cheapest route from A to G costs 22 mW (A, D, F, G), while A's route through the tree costs 23 mW (A, B, E, G). The live fleet shows the same gap: Pump A's cheapest route is Pump A, Moisture Sensor 1, Gateway at 21 mW over 2 hops, while its route through the tree is 44 mW over 6 hops, because the tree's 9 links cost 60 mW in total against 229 mW for all 24 links. The Topology page shows all three routes side by side so the difference is visible.

Self-healing

Failing Moisture Sensor 1 on the Topology page removes it from the graph. Pump A's cheapest route becomes Pump A, Moisture Sensor 2, Gateway at 22 mW, and the panel reports the 1 mW extra. Failing the Main Valve moves Valve East from 16 mW to 21 mW via Smart Meter 3. A node is also treated as down when the Part 2 watchdog flags it Disconnected after 60 seconds of silence, so real silence and simulated failures take the same path through the graph. A live node whose every route runs through failed nodes is reported as isolated, with the group it belongs to.

5. Challenges and how they were overcome

The hardest problems I experienced in Part 3 were about meaning rather than syntax: what a "range" is when timestamps repeat, what "the most power-efficient route" means for a network versus for one node, and what "down" means when a device simply goes quiet.

5.1 Time keys that repeat, and ranges that must include both ends

The archive stores readings by timestamp, but two readings from one device can share a timestamp, and a tree keyed on the timestamp alone would overwrite one. I made the key a pair of timestamp and ingestion sequence number, as the Part 2 timeline already did. A range query then has to include every reading at the boundary times, so the search bounds are padded with the smallest and largest sequence numbers. Tests cover both ends of a window and three readings with the same timestamp.

5.2 Keeping the tree valid while old readings are evicted

Retention removes the oldest reading once an archive reaches 20,000, and removing a node with two children is the easiest place to break an AVL tree. The tree exposes `ValidateInvariants`, which checks key order, stored heights and balance at every node. The tests call it after every insert in a randomised run, after removing half of 500 random keys, and compare 100 random range queries on 3,000 keys with a brute-force filter.

5.3 A spanning tree is not a shortest-path tree

The brief asks the MST to show "the most power-efficient routing path". It minimises the total power of the network, which is not the same as the cheapest route for one sensor. Treating them as one would have given wrong routes. I implemented Dijkstra for per-node routes and Prim for the network-wide tree, and made the page show both. A test on a worked graph proves the two can differ (23 mW through the tree against 22 mW cheapest), and on the live fleet Pump A's tree route is 44 mW against 21 mW.

5.4 Defining "down"

A node is down if an operator failed it or if the Part 2 watchdog has flagged it Disconnected. The first API smoke run that happened while the simulator was stopped showed every node down after 60 seconds, a spanning tree of 0 links and no routes. That was the design working, but my test had assumed live traffic. I fixed the test procedure to run the simulator and documented the behaviour in the README. A

related detail: Kruskal runs only over nodes the gateway can reach, so that when a failure splits the mesh it still reports the same total cost as Prim, which is what the randomised cross-check relies on.

5.5 A mesh that gives failures something to do

If every sensor linked straight to the gateway, failing a node would change nothing. Each new sensor links to its three cheapest existing neighbours, with costs from deployment locations and a stable hash, and I checked the generated fleet by hand: Pump A routes through Moisture Sensor 1 and Valve East through the Main Valve. The demo failures therefore produce visible reroutes, and because the hash is stable the same fleet always yields the same mesh.

5.6 Connecting the C# backend to the Blazor front end

The client reads its API address from `wwwroot/appsettings.json`, which points at `https://localhost:7086`. I had started the API on its plain `http` profile (port 5248), so the Command Stream page showed "Gateway unreachable" even though the API was healthy. Running the API on the `https` profile and trusting the development certificate fixed it. The lesson was to check the address and protocol the client is built against before suspecting the code. For the map itself I drew the mesh as inline SVG instead of adding a charting library: the gateway sits in the centre with sensors on a ring, switching to two alternating rings when the simulator registers about 20 devices, and the selected node, route type and history window are kept in `AppState` so they survive moving between pages.

5.7 Shared state across threads

The mesh graph is not thread-safe, and the API serves requests while a background worker ingests telemetry. `MeshTopologyService` takes one lock around every graph read or change, and the archive is updated under the same lock as the timeline, so a query never sees a half-updated tree.

6. Key learnings

The main lesson across the three parts is to choose a data structure from the question being asked of the data, not from habit. Smart-X ended up using a different structure for each question.

Question the gateway must answer	Structure	Part
Which sensor has this MAC address?	`Dictionary`, O(1) lookup	2
Show this device's newest readings in time order	`SortedDictionary` timeline	2
Is this alert already active?	`HashSet`	2
Which packet is processed next, and can a critical one jump the queue?	`Queue` and `PriorityQueue`	2
What did the operator just do, and can it be undone?	`Stack`	2
What did this device report between two times?	AVL tree	3

Question the gateway must answer	Structure	Part
What is the cheapest route to the gateway, and what is still connected?	Weighted graph with BFS, DFS, Dijkstra, Prim, Kruskal	3

Other learnings, each tied to something built in the project:

- **Test algorithms against an independent answer**

  For the mesh I did not only hand-check examples. Prim is tested against Kruskal on 100 random meshes, Dijkstra against Floyd-Warshall [6] on 30 random meshes with random failures, and the AVL tree against a brute-force filter. Two different methods agreeing is stronger evidence than one method matching an expected value I typed myself.

- **Recursion is clear but needs a bound**

  The AVL insert, delete and range search, the depth-first search, the path-to-gateway walk and the Part 2 deployment-location validation are all recursive. The tree's recursion is safe because its height is logarithmic, and the mesh is small, but a depth-first search on a very large mesh should use an explicit stack. I now check what bounds the depth before choosing recursion.

- **Operator overloading earns its place when it removes noise**

  `LinkCost` lets Dijkstra read `cost + linkCost` and `candidate < known`, and Part 2's `NumericTelemetryPacket` overloads `-` so the pipeline can report "power jumped +N W" from two packets directly.

- **Design failure behaviour, not only the happy path.**

  Treating a silent device as a down node meant simulated and real failures share one code path, and the page handles an unreachable API with a banner instead of a crash.

- **Small commits and a build that runs on every push**

  The repository now has a GitHub Actions workflow that restores, builds and tests on each push, and history in small steps with plain messages, which made it easy to see where a change came from while debugging.

- **Client state belongs in one place**

  Blazor discards a page's state when you navigate away, so the selected node, route type and history window live in `AppState`. Polling uses a `PeriodicTimer` that is cancelled when the page is disposed, which stops leaked requests.

7. Testing and verification

The solution builds cleanly on .NET 10, all 132 automated tests pass, and a scripted check of the running API passes 46 of 46 steps. The checks were run on a fresh clone of the repository.

Layer	What it checks	Result
Unit tests (xUnit)	Shared models and anomaly rules, alert tracker, device registry, AVL tree, min-heap, `LinkCost`, mesh graph, history archive, mesh service	132 of 132 passed
Randomised cross-checks	Prim against Kruskal (100 meshes), Dijkstra against Floyd-Warshall (30 meshes with failures), AVL range against a brute-force filter	Passed
API smoke script (running API and simulator)	Archive range and benchmark, spanning tree, routes, failure, reroute, restore, error codes (400 and 404), Part 2 endpoints still working	46 of 46 passed
Part 2 regression in the browser	Dashboard, sensors and command stream pages load and update live with the simulator running	Confirmed
Continuous integration	GitHub Actions restores, builds and runs the tests on every push	Configured

What the API script proved, with numbers from the run: a 10-minute window returned about 60 readings and touched 74 of 8,690 tree nodes; the tree beat the scan by 33 to 57 times across runs; the mesh had 10 nodes, 24 links and a 9-link spanning tree of 60 mW against 229 mW for all links; failing Moisture Sensor 1 rerouted Pump A from 21 mW to 22 mW through Moisture Sensor 2; failing the Main Valve rerouted Valve East to 21 mW; restoring both returned every route to normal.

Known build warnings, none of which affect behaviour: the build reports advisory NU1903 for `Microsoft.OpenApi` 2.0.0, which should be upgraded when a patched version is available for the target framework, and warning CS1718 in the operator tests, where a value is deliberately compared with itself to exercise the comparison operators.

Figure 1 shows the healthy mesh with Pump A selected: nine sensors and the gateway joined by 24 links, all nodes up, a 60 mW spanning tree out of 229 mW, and the cheapest route Pump A → Moisture Sensor 1 → Gateway at 2 hops and 21 mW.

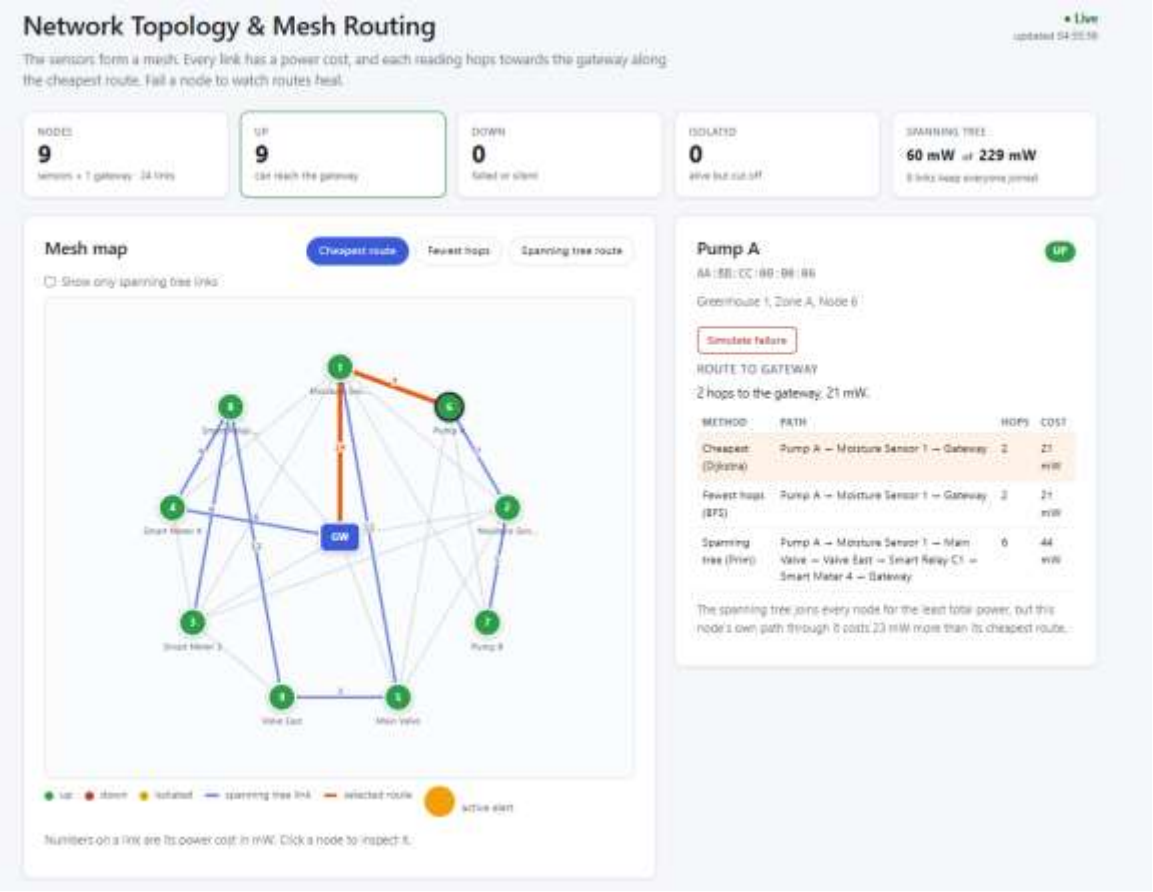
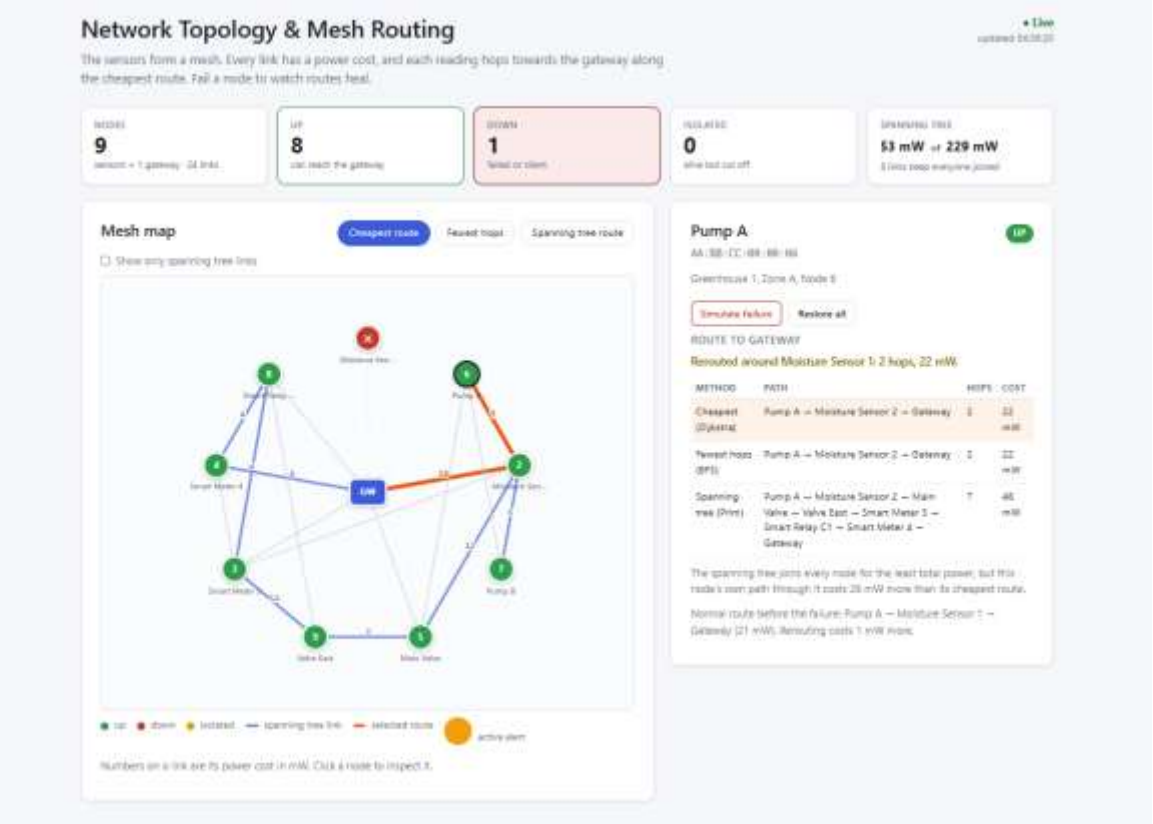


Figure 2 shows the same view after Moisture Sensor 1 is failed. The page marks the node down, rebuilds the graph and reroutes Pump A through Moisture Sensor 2 (2 hops, 22 mW, 1 mW more than before). The spanning tree shrinks to 8 links and 53 mW.



8. Technology recommendations and future scaling

Smart-X today keeps everything in memory, receives telemetry over HTTP from simulated devices, and refreshes the browser by polling every 1.5 to 2 seconds. The recommendations below fix those limits in order of value, and each one fits the existing .NET 10 stack.

#	Recommendation	Limit in Smart-X today	Benefit	Fit with .NET
1	**MQTT broker with MQTTnet**	Devices are simulated over HTTP POST. Silence is found by a 60-second watchdog.	MQTT is a publish/subscribe protocol that suits resource-constrained devices on unreliable or low-bandwidth links [2]. Its specification defines delivery levels and a last-will message that a broker can publish when a device drops, which would mark a node down at once [7]. The same	MQTTnet is a .NET client and server library, so the ingestion pipeline can subscribe without changing the queues behind it.

#	Recommendation	Limit in Smart-X today	Benefit	Fit with .NET
			topics let Home Assistant consume the data.	
2	**TimescaleDB on PostgreSQL**	History lives in memory and is lost on restart. Archive size is capped at 20,000 per device.	Hypertables partition time-series data by time, which suits range queries and dropping old data. A survey of time series management systems notes that such data keeps growing as monitoring and automation spread [8]. The AVL archive stays as the fast in-memory index for recent windows.	Npgsql and Entity Framework Core work with PostgreSQL, so `TelemetryHistory` can write through to the database.
3	**SignalR for live updates**	The Topology and Command Stream pages poll the API every 1.5 to 2 seconds.	The server pushes a change the moment it happens, so a failed node turns red immediately and idle clients cost no requests.	SignalR is part of ASP.NET Core, and Blazor WebAssembly has a ready client package.
4	**OpenTelemetry metrics and traces**	Queue depth, latency and duplicate counts are visible only inside the app.	OpenTelemetry gives a vendor-neutral way to collect traces and metrics from a .NET application, so queue depth and latency could be exported to a monitoring tool and an operator alerted when the priority queue backs up.	The .NET SDK and ASP.NET Core instrumentation attach with a few lines in `Program.cs`.
5	**Authentication, rate limiting and a tighter CORS policy**	The API has no sign-in, and CORS allows any origin. Anyone who can reach it can fail a node or send commands.	Operators and devices get identities, abusive clients are throttled, and only the real client origin is trusted. Surveys of IoT middleware treat authentication and access control as core security requirements [9].	ASP.NET Core includes rate-limiting middleware.
6	**Azure IoT Hub with device provisioning**	One shared API accepts any device id.	Per-device identity and secure device-to-cloud messaging for thousands of devices, with routing to storage and analytics.	Azure IoT device and service SDKs exist for .NET.

#	Recommendation	Limit in Smart-X today	Benefit	Fit with .NET
7	**Extend GitHub Actions into a CD pipeline**	CI builds and tests, and Docker images are built by hand.	Every merge builds container images and deploys to a test environment, so a regression is caught before a demo.	Workflows already run `dotnet build` and `dotnet test`. The existing Dockerfiles can be reused for Azure Container Apps or Kubernetes.
8	**LLM API to summarise alerts**	Operators read raw alert lists and suggestions.	A daily plain-language summary of alerts and insights saves reading time. It should be advisory and reviewed by a person, with device data kept to the minimum needed.	Called from a hosted background service through an HTTP client.

Scaling the Part 3 structures

The mesh is recomputed on each request, at O((V + E) log V) for routes, which is instant for hundreds of nodes. Beyond thousands, routes and the spanning tree should be cached and recomputed only when a node's state or a link changes, and the archive should hand cold history to the database while the AVL tree keeps the recent window. A larger mesh would also replace the recursive depth-first search with an explicit stack.

9. Feedback incorporated and AI use

Feedback

The two Part 1 feedback items were operator overloading and recursion. Both were addressed in Part 2 (`TelemetryPacket<T>` comparison operators and `NumericTelemetryPacket<T>` arithmetic operators; recursive deployment-location validation and zone-tree roll-ups) and reused in Part 3 (`LinkCost` operators; recursion in the AVL tree, depth-first search and path-to-gateway walk).

AI use

Claude AI was used in the code to assist with complex calculation and algorithm generation, like the one for suggested actions. I also used Claude heavily to create and troubleshoot the simulation tool for the project, so that the data stays realistic and the tool still works correctly when an abnormality is pushed. Claude was also used in the frontend to assist with overall layout. I wrote the main code myself (st10440432) and used Claude to troubleshoot errors as they arose. In Part 3 Claude drafted Skeleton code, helped troubleshoot and test. I directed the scope and the order of the work, built and ran everything on my own machine, tested the results, and reviewed the output before submitting it.

In this document, Claude AI was used as a guide for the structure and skeleton, and to review my arguments for cohesion. It was also used to gather accurate technical values, which I then incorporated into my writing as supporting evidence.

10. References

[1] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: A survey," *Comput. Netw.*, vol. 38, no. 4, pp. 393–422, 2002. [Online]. Available: https://ianakyildiz.com/bwn/surveys/sensornets.pdf

[2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 4, pp. 2347–2376, 2015. [Online]. Available: https://ianakyildiz.com/bwn/SENSOR20/reading/IoT_survey.pdf

[3] D. M. Mount, "CMSC 420: Lecture 5, AVL trees," Dept. Comput. Sci., Univ. Maryland, College Park, MD, USA, Lecture notes, 2020. [Online]. Available: https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Lects/lect05-avl.pdf

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959. [Online]. Available: https://courses.cs.cornell.edu/cs2110/2017sp/online/shortestPath/1959dijkstra.pdf

[5] J. Erickson, "Minimum spanning trees," in *Algorithms*, ch. 7, Univ. Illinois Urbana-Champaign, Urbana, IL, USA, Lecture notes, 2019. [Online]. Available: https://courses.grainger.illinois.edu/cs374/sp2019/notes/07-mst.pdf

[6] J. Erickson, "All-pairs shortest paths," in *Algorithms*, ch. 9, Univ. Illinois Urbana-Champaign, Urbana, IL, USA, Lecture notes, 2021. [Online]. Available: https://courses.grainger.illinois.edu/cs374/fa2021/A/notes/09-apsp.pdf

[7] OASIS, "MQTT version 5.0," OASIS Standard, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[8] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "Time series management systems: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 11, pp. 2581–2600, 2017. [Online]. Available: https://arxiv.org/abs/1710.01077

[9] P. Fremantle and P. Scott, "A survey of secure middleware for the Internet of Things," *PeerJ Comput. Sci.*, vol. 3, p. e114, 2017. [Online]. Available: https://peerj.com/articles/cs-114